C Programming & Data Structures
Pg-255

Page 12
Date

13/9/20

- 7 concepts C programming
- 7 " Data Structures
- 3 language history
- Name 2 reasons why C
- Portability comparison C & Assembly
- Less liner of code eg
- 4 features of C eg
- High level V/s lower level
- Eg languages
- why C middle level language? -}
- 6-7 libraries eg
- 5-6 built in functions
- simple code
- Comments syntax
- #include <stdio.h> is —
- Dig of preprocessor
- Preprocessor - 3
- Now explain #include <stdio.h>
- stdio.h expand
- ↓ 3 points
- Syntax of a function
- why diff files? - diagram
- why linker used not preprocessor
- Hello world program
- Run " " with full changes
- a) Comment #include <stdio.h>
- b) Remove semicolon after printf
- c) Remove int return type from main
- d) Return any integer
- e) Replace 0 with 'a'
- f) Remove semicolon after return.

Scanned with CamScanner

Page
Date

Variable through glass water eg:
Note
Declaration
2 properties
Defn of a variable
Most of time
Eg of declaration
Eg of glass
2 notes
Initialisation

```
#include <stdio.h>
int main() {
  int var=3;
  var=4;
  printf("%d", var);
}
```

Assign variable to another variable

```
int main() {
  int var1=3;
  int var2;
  var2=var1;
  printf("%d", var2);
}
```

Define variable in a single line

```
#include<stdio.h>
int main() {
  int var1, var2, var3;
var1 = var2=var3=4;
printf("%d", var1);
printf("%d", var2);
printf("%d", var3);
}
```

Scanned with CamScanner

Variable name - general convention
Rule #1
eg
Rule #2
Rule #3
eg
Generally
Symbolic names meaning
Rule #4
Rule #5
eg
Rule #6
eg - wrong
eg // Comment
Tip
Printf
Printf ("Nesco Academy") → conclusion
" " ("%d", var), → " "
" " ("%d %d", var1, var2) → " "
" " ("%d %d %d", var1, var2); "

```
#include <stdio.h>
int main() {
    int two=2; three=3; six=6;
    int result = (two+three)*six/three;
    printf("I am new to programming but I
know how to write arithmetic
expressions like this: (%d+%d)*%d/%d
    two, three, six, three);
printf("and the result of this
expression is: %d", result);
}
```

Scanned with CamScanner

1st fundamental data type
Size of integer =
2 bytes =
4 bytes =
How to know size of integer?
Operator syntax
Code of size of
Why size of (4 more types of int)
What is range
Range of {1, 2, 3, 4}
What is decimal # system
Other name
Range
Eg 568
Binary # system
Other name
Range
Eg 1100
Range of 4 bit data
General formula to calculate max value
Range of 2 bytes (2 types of 2 bytes)
Unsigned range - formula
range
Signed range - formal
range
Name of range
Unsigned range - 4 bytes
Signed " - 4 bytes
Name 4 modifiers
long & short
By default
Unsigned syntax
Only — value

Find range programatically - signed int
unsigned "
signed short int
Unsigned " "
How placeholders changed
Side note if else
Find range programatically - Signed long
unsigned "
signed long long int
Unsigned " "
Summary - 1
Summary - 6 (placeholders)
What if I exceed range - 2 subtypes
Code for exceeding unsigned range
Let's understand 3 bit for exceeding range
So general for 3 bits
" "
Exceeding range signed int
2nd data type - name
ASCII values are till
" " " of 0
A
a
Quotes single/double
Placeholder
Code
Size & range of characters signed
unsigned
-128
+128
-127
+129
General conclusion

Scanned with CamScanner

Program to verify
0 → 127 — ASCII
0 → 128 → 255 "
1 - 128 → -1 - "
Datatype to represent fraction - 3
Float bytes
Double "
long " "
Representation - Float
" Double
" long
types of "
Fixed point representation general
Min value
Max "
Cannot represent —) why
Floating point representation - general
M exponent
Formula
Min value
Max "
Explain why 3 diff type - program
O/P - digits
String placeholder
Program for diff b/w float & double
4/9
4.0/9.0
What is the diff of following -

```
#include<stdio.h>
int main(){
printf("%d",printf("%s","Hello World"));
}
```

Scanned with CamScanner

Page 68
Date

Q. %0s is used for
a) Printf other function
b) return
c) I/P of int main() {
printf("%10s", "Hello"); }

d) O/P of int main() {
char c=255;
c+=10;
printf("%d", c); }
a) 265
b) Something ASCII
c) 7
d) 9

Q. which is correct
i) signed int i;
ii) i;
iii) unsigned i;
iv) long i;
v) long int i;
vi) long i;
a) I, IV
b) I
c) All
d) IV, V, VI

Q. what happens whenever int not mentioned
int main() {
unsigned i=1;
int j=-4;
printf("%u", i+j); }
a) garbage
b) -3
c) Integer value depends on m/c & compiler
d) None

Q. Scope of variables
Q. scope meaning-2
Q. Strict definition
Q. Basic miracle of scoping { { } }

h) { }
}

Scanned with CamScanner

```
#include <stdio.h>
int main() {
int var=3;
{ int var=4;
printf("%d\n", var);
}
printf("%d", var);
return 0;
}
```

```
#include <stdio.h>
int fun();
int var = 10;
int main() {
int var=3;
printf("%d\n", var);
fun();
}
int fun() {
printf("%d", var);
}
```

- 2 storing modifier
- Auto modifier
- Syntax
- Eg Code-2
- When scope of auto ends
- What is we don't write auto
- Take away-1
- Code
- Take away-2
- Code
- extern modifier syntax
- Diff from auto

Scanned with CamScanner

Q. Extern expand
Q. When used
Q. If code of extern1.c, extern2.c
Q. What if we write in extern1.c
extern int a;
Q. Why allowed
Q. What if we delete extern2.c &
in extern1.c we have full code
#include <stdio.h>
int a=9;
int main(){
extern int a;
printf("%d",a);
return 0;
}
Q. Meaning of extern int a=9;
Q. What compiler believes
Q. Modifier registers pyramid
Q. Label primary
Q. " auxiliary
Q. Access time
Q. Register Memory
Q. Mnemonic Caper
Q. Syntax of register
Q. 2 things to do
Q. 2 imp points
Q. Modifier static()
#include <stdio.h>
#include <stdlib.h>
int increment();
int main(){
int value; value=increment(); value=increment

Scanned with CamScanner

value = increment();
printf("%d", value);
return 0; }
int increment() {
static int count;
count += 1;
return count;
}

Q- Takeaway -1 (Static V/s auto)
Q- Takeaway -2
Q- " -3
Q- what is constant
Q- Once defined
Q- 2 type of defining constants
Q- Syntax of #define
Q- Who's job?
Q- Also called as
Q- #include <stdio.h>
#define PI 3.14159
int main() {
printf("%.5f", PI);
return 0;
}

Q- Takeaways - just name 6
Q- 1st explain
Q- 2nd "
Q- 3rd "
Q- " eg
Q- 4th explain
Q- " eg
Q- 5th explain
Q- " eg

Scanned with CamScanner

Page 12
Date

6th - name - 2

11 eg

Using cont, syntax

Note

Code

O/P of int main(){
int var = 052;
printf("%d", var);
return 0;
}

a) 52
b) 56
c) 42
d) compiler error

Always remember

What if we write %0 in above ques in place of %d:

O/P of #include<stdio.h>
#define STRING "%s\n"
#define NESO "Welcome to Neso Academy!"
int main(){
printf(STRING, NESO);
return 0;
}

Standard I/P function

Expand Scanf

What accept

format specifier in scanf

%c for

%s for

& pronunciation

scanf code simple

why &? - 2

& is also called _____ operator

1. To store I/P data, what scanf need

&var = _____ of var

Program to add 2 Numbers - scanf

Scanned with CamScanner

Page
Date

Q. #include<stdio.h>
int main() {
int var = 0x43ff;
printf("%x", var);
}

Q. Diff b/w %x & %X

Q. o/p of #include<stdio.h>
static int i;
i=27;
int main(){
static int i;
printf("%d", i);
return 0;
}

a) 27
b) 0
c) No o/p
d) None

Q. Draw memory layout of C program
Q. Uninitialized
Q. Command line
Q. Initialize data
Q. Text/code segment
Q. 2 memory segments
Q. Explain
Q. Note →
Q. where stored?
Q. Now am the question
Q. #include<stdio.h>
static int i;
i=27;
i=45;
int main(){
printf("%d", i);
return 0;
}

Scanned with CamScanner

Q. #include<stdio.h>
static int i;
static int i=27;
int main(){
i=45;
printf("%d", i);
return 0;
}

Q. Conclusion

Q. Name Types of operators in C
Q. Arithmetic - 5
Q. Inc/Dec - 2
Q. Relational Operators - 5
Q. Logical - 3
Q. Bitwise - 6
Q. Assignment
Q. Other operators - 4, 5
Q. Arithmetic operators are unary/binary/ternary
Q. eg code
Q. Precedence & associativity arithmetic
Q. when Associativity is used operators
Q. Eg #include<stdio.h>
int main(){
int a=2, b=3, c=4, d=5;
printf("a*b/c = %d\n", a*b/c);
printf("a+b-c = %d\n", a+b-c);
printf("a+b*d-c = %d\n", a+b*d-c);
}

Q. Why we have to use %%d
Q. Inc dec operator function
Q. Increment code
Q. Decrement code same as same as

Scanned with CamScanner

Q. Increment / Decrement are unary / binary
Q. 2 type of increment
Q. 2 " " decrement
Q. Rule of rvalue / lvalue in increment / decrement
Q. lvalue expand
Q. 4
Q. Rvalue - expand
Q. Lvalue - 3
Q. a++ (a+b)++
a = a+1 (a+b) = (a+b)+1
Q. why error by compiler
Q. Diff b/w Pre & post increment
Q. O/P of #include <stdio.h>
int main() {
int a=4, b=3;
printf("%d", a++b);
return 0;
}
Q. Token generation - First phase in compiler
Q. What does lexical analyzer do
Q. what is token
Q. Eg of token nab
Q. lexical analyzer for int a=5;
Q. How " " analyze
Q. lexical analyzer for a+++5
Q. Post increment / decrement in context of eqn
Q. Pre increment / decrement in context of eqn
Q. O/P of #include <stdio.h>
int main() {
int a=4, b=3;
printf("%d", a+++b);

Scanned with CamScanner

Q. o/p of a+++++b
Q. a++ + ++b
Q. Relational operator - all
Q. Used for -
Q. What relational operator return
Q. logical operator - 3
Q. && and ||
Q. when true, when false
Q. ! operator usage
Q. Concept of short circuit in logical operator
Q. Short circuit in && - defn

```
#include<stdio.h>
int main(){
int a=5, b=3;
int incr;
incr = (a<b) && (b++);
printf("%d\n", incr);
printf("%d", b);
return 0;
}
```

Q. Short circuit in || - defn

```
#include<stdio.h>
int main(){
int a=5, b=3;
int incr;
incr = (a>b) || (b++);
printf("%d\n", incr);
printf("%d", b);
return 0;
}
```

Scanned with CamScanner

Page
Date

Introduction to bitwise operators - 6
How performed - bitwise & - 3
Eg of 0111 & 0100
Bitwise OR operator - 3 7|4
Eg of 0111 & 0100 7|4
Bitwise NOT operator - 3
Eg ~7
Diff b/w bitwise & logical

```
#include<stdio.h>
int main(){
  char x=1, y=2;
if(x&y)
  printf("Result of x&y is1")
if(x&&y)
  printf("Result of x&&y is 1");
}
```

Left shift operator
First operand << Second operand
Imp point - 1

```
#include<stdio.h>
int main(){
  char var=3;
  printf("%d", var<<1);
  return 0;
}
```

Left shifting is equivalent to ___
Right " operator
First operand >> Second operand
Imp points - 2
General
Bitwise XOR operator

Scanned with CamScanner

Q. Simply OR or inclusive OR-2
A: Inclusive OR-2
Q. Bitwise XOR
A: Result of XOR is 1 & 0, when
Q. eg. 7 0 1 1 1
4 0 1 0 0 7^4
Q. Assignment operator.
Q. 2 values
A: copy — to —
Q. var = 5;
A: Shorthand assignment operators
14/9/20 Q. Conditional operators - Syntax
Pg-30 Q. unary / binary / ternary
Q. why required?
Q. Write chor result;
if (marks > 33),
result = 'P';
else
result = 'F';
After with ternary
Q. what 2 things returned
A: expression & result
Q. Quick sort
Q. #include <stdio.h>
int main() {
int var = 75;
int var2 = 56;
int num;
num = sizeof(var) ? (var2 > 23 ? ((var == 75) ? 'A' : 0) : 0) : 0);
printf("%d\n", num);
}

Scanned with CamScanner

Page
Date

Q. Comma operator — Separator
Q. , , , , , , — operators
Q. int a = (3, 4, 8);
printf("%d", a);
Q. int var = (printf("%s\n", "Hello!"), 5);

Q. Precedence of C
Q. int a;
a = 3, 4, 8;
printf("%d", a);
Q. int a = 3, 4, 8;
printf("%d", a);
Q.
int a;
a = (3, 4, 8);
printf("%d", a);
int a = (3, 4, 8);
printf("%d", a);

Q. #include <stdio.h>
int main(){
int var;
int num;
num = (var = 15, var += 3 5);
printf("%d\n", var);
}

Q. Precedence of operators
Q. Associativity of operators — 2
Q. Draw table of Associativity & Precedence
Q. comma
Q. parenthesis in function calls
Q. Eg

Scanned with CamScanner

Q: Operator precendence as compared to ()
Q: () treated in int var = fun();
Q: Member access operator.
Q: Postfix Increment/decrement
Q: Precedence of post, Prefix
Q: Associativity of post, Prefix
Q: Unary meaning
Q: Unary (type) is ___
Q: Imp fact -1

```
int main(){
a=fun1()+fun2();
printf("%d", a);
return 0;
}

int fun1(){
printf("Neso");
return 1;
}

int fun2(){
printf("Academy");
return 1;
}
```

Q: Which is called first fun1() or fun2()?
Q: When associativity comes into action.
Q: So OP
Q: Same precedence operator have same associativity (T/F)

```
#include<stdio.h>
int main(){
int a=10, b=20, c=30, d=40;
if(a<=b==d>c)
printf("TRUE");
else
printf("FALSE");
```

OP of

```
#include<stdio.h>
int main(){
int i=5;
int var = sizeof(i++);
printf("%d %d", i, var);
}
```

a) 5, 4
b) 6, 4
c) 5, 8
d) Compile error

Q:- What is C standard

Q:- Acc to C99 standard

Q:- C99 newer version

Q:- C11

Q:- o/p of int a=1;
int b=1;
int c=++a || b++;
int d=b-- && --a;
printf("%d %d %d %d", d, c, b, a);

a) 1111 b) 1001 c) 1001 d) 1101

Q:- #include <stdio.h>
int var=5;
int main(){
int var=var;
printf("%d", var);
}

a) 5 b) Compiler error c) Garbage value d) None

Q:- Is it necessary to put else part?

Q:- q1
int n=4;
if(n!=4)
printf("n is not 4");

q2
int n=4;
if(n!=4)
printf("n is not 4");
else
printf("n is 4");

Q:- Switch - Replacement to Replace int x=2;
if(x==1)
printf("x(1)");
else if(x==2)
printf("x(2)");
else if(x==3)
printf("x(3)");

Scanned with CamScanner

else
printf("x is a # other than 1, 2 & 3");

int x = 2;
Make by switch
Q: Default if's —
Q: what if no break is there
Q: Facts related to switch — 4
Q: Allwed code
Q: Not // code
Q. Float —
Q: 4th code
Q: // not //
Q: where default can be placed
Q: Importance of breaks, while, for
Q: Diff b/w while & do while — 2
Q: when should I prefer do while over while — Eg code
Q: Loop control statements — 2
Q: WAP to allow user to enter integer until he/she enters a -ve number or zero:
Q: continue
Q: diff with break
Q: WAP to print all odd # from 1 to 20
Q: How many times Hello would be printed

```
#include<stdio.h>
int main(){
  int i=10;
for(;i;i>>=1)
printf("Hello World");
return 0;
}
```

a) 10
b) 1
c) infinite
d) none

Scanned with CamScanner

Page
Date

Q. O/P of #include<stdio.h>
int main() {
int i;
for(i=0; i<20; i++){
switch(i){
case 0: i+=5;
case 1: i+=2;
case 5: i+=4;
}
printf("%d", i);
}

a) 5 10 15 20
b) 7 12 17 22
c) Compilation error
d) 16 21

Q. #include<stdio.h>
int main(){
int i=0;
for(printf("one\n"); i<3 && printf("."); i++){
printf("Hi\n");
}}

Q. O/P of #include<stdio.h>
int main(){
unsigned int i=500;
while(i++ != 0);
printf("%d", i);
return 0;
}

a) 0
b) 1
c) Infinite loop
d) Compilation error

Q. What to note in above q (\n)?
Idea for pyramid of stars
Simple O/P
Q. Rows ws = — column
Code

Scanned with CamScanner

- what is pallindrome #
- Logic for " "
- Armstrong number
- Algo
- Step 1
- Step 2
- Step 3
- Whole code
- what is strong #
- Algo
- Program to find strong #?
- 2 steps
- WAP to find if # is prime
- How to check - 2nd (New algo)
- General
- Code
- sqrt meaning
- where available
- Syntax
- Ceil function
- where available
- Syntax
- Add 2 # without using '+' operator
- Code
- Extend to -ve integers
- Idea + 2
- Draw Half adder Truth table
- Code logic
- Draw all iterations for 7, 2
- Draw fibonacci spiral
- Print Fibonacci series upto n number of terms

Scanned with CamScanner

Page
Date

Q- What is Floyd's triangle?
Q- Eg floyd Δ with 5 rows
Q- Code to print floyd's Δ of n rows
Q- Binary to decimal code
Q- Power of an integer eg
Q- Code for +ve power
Q- " for -ve "
Q- Defn of leap year
Q- 3 steps to check leap year
Q- Code
Q- Perfect number - defn
Q- eg
Q- Introduction to function - defn
Q- Syntax
Q- Why functions - 2
Q- eg
Q- Code to get area of rect
Q- Function declaration other name
Q- " " " meaning
Q- Properties - 5
Q- Imp point - 1
Q- " - 2
Q- Code of func declaration
Q- Is it always necessary to declare before use
Q- Code
Q- What happens if we call function before defining / declaring
Q- Func defn ??
Q- eg
Q- Code to add
Q- Imp to mention in defn
Q- Diff b/w argument & Parameter

Scanned with CamScanner

Q. Other name: Parameter
: Argument
Q. Go code
Q. What is call by value
Q. Swap program
Q. Call by reference - syntax
Q. Swap program
Q. int func(int num){
int count=0;
while(num){
count++; num>>=1;}
return count;}
The value of fun(435) is
Q. O/P of int fun(){
static int num=16;
return num--;}
int main(){
for(fun(); fun(); fun())
printf("%d", fun());
return 0;
}
Q. By default functions are ___ in C
Q. How to make it non-global
Q. Imp takeaways - 2
Q. Draw memory block diy
Q. Explain about the stack component - 3
Q. Other name stack
Q. Draw callstack for
main(){fun1();}
fun1(){fun2();}
fun2(){fun3();}
fun3(){return;}
Q. In reality

q. Infinite loop
b. 13 10 7 4
c. 14 11 8 5 2
d. 15 12 8 5 2

Scanned with CamScanner

Activation record-3

Q-1. eg

Q- Why scoping

A- eg

Q- What is static scoping?

A- Memory diy

```
int a=10, b=20
int fun() {
  int a=5; {
int c;
  c=b/a;
printf("%d", c);
}}
```

Q:
```
int fun1(int);
int fun2(int);
int a=5;
int main(){
int a=10;
  a=fun1(a);
printf("%d", a);}
int fun1(int b){
   b=b+10;
   b=fun2(b);
   return b;}
int fun2(int b){
   int c;
   c=a+b;
   return c;
}
```

Scanned with CamScanner

Page
Date 88

Q What is dynamic scoping
Q Execute same eg
Q Scoping in most languages eg
Q " " " C
Q Dynamic " eg languages
Q Can we see programatically the scoping in C
Q Which language support both scoping
Q eg code
Q local meaning
Q my "
Q O/P for

```
$x = 50;
sub fun2 {
  return $x;
}
sub fun1 {
  local $x = 10;  -(1)
  my $y = fun2();
  return $y;
}
print fun1();
```

Q Solve by my instead of local in (1)
Q Consider the program below in a hypothetical programming language which allows global variables & a choice of static or dynamic scoping

```
int i;
program main() {
  i = 10;
  call f();
}
procedure f() {
  int i = 20; call g();
}
procedure g() {
  print i;
}
```

Let x and y be value printed under static & dynamic scoping, then
a) x=10, y=10
b) x=20, y=10
c) x=10, y=20
d) x=20, y=20

Scanned with CamScanner

Def^n of recursion
eg
Recursion can be seen/visualised through
```
int fun(int n){
    if(n==-1)
      return 1;
  else
    return 1+fun(n-1);
}
int main(){
    int n=3;
    printf("%d", fun(n));
    return 0;
}
```
How to write a recursive program-2
Calculate factorial of a # using recursion
4 type of recursion
Direct recursion-def^n
" " structure
Indirect recursion-def^n
" " structure
WAP to print numbers from 1 to 10 in such a way that when number is odd, add 1 & when number is even, subtract 1.
Tail recursion def^n
eg
Call stack
Evaluation ahead of backtrack
Hence,
Non-tail recursion def^n
eg
Diff here
Why opposite

Scanned with CamScanner

Q) Tell if int fun (int n) {
if (n==1) return 0;
else return 1+fun(n/2); }
Is recursive or not? Also tell tail or non tail recursive

Q) 1 advantage of recursion over iteration
Q) 1 " " iteration over recursion
Q) Factorial of a # recursion
iteration

Q) void get(int n) { if (n<1) return;
get(n-1); get(n-2); printf("%d", n); }
If get(6) is called in main, how many times will get function invoked
a) 15 b) 25 c) 35 d) 45

Q) Determine how many # of times * will be printed on screen
void fun(int n) {
int i=0;
if (n>1)
fun(n-1);
for(i=0; i<n; i++)
printf("*");
}
a) n
b) n(n+1)/2
c) n+n
d) None

Q) int fun(int n) {
int x=1, k;
if (n==1) return x;
for(k=1; k<n; ++k)
x = x + fun(k) * fun(n-k);
return x;
}
fun(5) = ?
a) 0
b) 26
c) 51
d) 71

Scanned with CamScanner

Page
Date

Grate 2017

Q1 int f(int j){
static int i=50;
int k;
if(i==j){
printf("something");
k=f(i);
return 0;
}
else return 0;
}

TRUE?
a) The function return 0 for all j.
b) function print "something" for all j
c) The function return 0 when j=50
d) function will exhaust the entire stack as run into an infinite loop when j=50

Gate 2016

Q2 What will be o/p of following pseudocode when parameters are passed by reference & dynamic scoping is assumed

a=3
void n(x){
x=x+a;
print(x);
}
void m(y){
a=1;
a=y-a;
n(a);
print(a);
}
void main(){
m(a);
}

a) 6, 2
b) 6, 6
c) 4, 2
d) 4, 4

Q3 O/P of void count(int n){
static int d=1;
printf("%d", n);
printf("%d", d);
d++;
if(n>1) count(n-1);
printf("%d", d);
}

count(3) is
a) 3 2 2 1 3 4 4
b) 3 1 2 1 1 2 2 2
c) 3 1 2 2 1 3 4
d) 3 1 2 1 1 1 2

Scanned with CamScanner

Q. find true a) Static functions are global
b) Static func are restricted to files where functions declared
c) No concept of static functions
d) None

Q. In C it is mandatory to declare a function before use (T/F).

Q. In parameters are always a) Passed by value
b) Passed by reference. c) Both d) None

Q. Assuming int size is 4 bytes, what is going to happen when following program run?

```
#include<stdio.h>
int main()
{
  printf("Hi!\n");
  main();
  return 0;
}
```

a) We can't use main inside main
b) Hi will be printed 2^{31} times i.e. $2^{31}-1$
c) It will take overflow
d) Hi only once

Q. Def of array
Q. what is data structure?
Q. Eg
Q. Dig
Q. Explain # of data values - eg
Q. Data type of array elements
Q. Eg of valid & invalid data type array
Q. What is one-d array
Q. Syntax of 1-d array
Q. Explain declaration, definition
Q. Explain compiler
Q. Int a[5.67]

Scanned with CamScanner

Q- Tell if valid
(i) int arr[5];
(ii) int arr[5+5];
(iii) int arr[5+3];
(iv) int arr[-5];
(v) int a; int arr[a=21][7];

Q- Best practice
Q- Syntax
Q- Why & code
Q- Future changes in both codes
Q- How to access an array element
Q- Index range
Q- Last element = a[?]
Q- Initializing 1D array — Method 1
2
3
4
Q- Which methods not preferred
Q- What if # of elements are less than the length specified?
eg
Q- Alternate for → int arr[10];
for(i=0; i<10; i++)
arr[i]=0
Q- Can we code arr[10]={};
→ What if more elements than left
Q- What if we want
int arr[10]={1,0,0,0,0,2,3,12,0,0};
Q- What is designated initialization
, designator
Q- Work in designated

Scanned with CamScanner

- 2 advantages
- Diff b/w int a[15] = {[0]=1, [5]=2};
  int a[15] = {[5]=2, [0]=1};
- what if I don't mention length in designated array?
- Size of int a[] = {[5]=90, [20]=4, [1]=45, [49]=78};
- Is int a[] = {1, 7, 5, [5]=90, 6, [8]=4} Valid.
- If valid, where in non-designated fun.
- What if there is a clash
- Meaning of int a[] = {1, 2, 3, [2]=4, [6]=45}
- WAP to print 34 56 54 32 67 89 90 32 21 in reverse order?
- WAP to check whether any of the digits in a number appear more than once
- How to get array size using sizeof operator
- Mathematical formula
- What are multi-dimensional arrays
- General form of ... array
- Eg 2-d array
- " 3-d "
- How size of multi-D calculated
- Eg size of a[10][20], a[4][10][20]
- Syntax 2-d array
- How to visualize
- Initialization method I
- " " better II
- How to access 2D array elements
- Print 2D array
- Visualizing 3D array
- Ie arr[2][3][3]

Page
Date

Q- Initialize 3D array - methods
both 1"
Q- Printing elements of 3D array
Q- WAP that reads 5x5 array of integers & then print the row sum & column sum
Q- Basics of matrix multiplication - 1
Q- " " " - 2
Q- Code of " "
Q- What are constant arrays in C
Q- Eg
Q- Advantages - 2
Q- Variable length arrays in C
Q- Eg
Q- Advantages 3
Q- Tell if valid
a) int a[3+i+5];
b) int a[k17+2];
c) int a[i+j];

Q- Points to be noted - 2
Q- Pointer Imp point - 1
Q- " " " - 2
Q- General syntax for declaring pointer
Q- Data type in pointer usage
Q- Initialize - 3
Q- Eg of Initialize
Q- Pg of "
Q- 3 names of value of operator
Q- Where used
Q- Eg

Scanned with CamScanner

Q: Where dereferenced pointer used, eg
Q: Caution -1
Q: 1 2
Q: Which fault occurs
Q: Pointer assignment eg
Q: Through eg prove that q=p is not same as *q=*p. Visual Sc also
Q: Using pointers find largest & smallest element in an array
Q: Returns pointer (element at mid-array)
Q: int* return type is —
Q: Caution 1
Q: Eg
Q: Why illegal
Q: Consider int *p=&i, p=&i, first statement is the declaration & second is simple assignment statement, why isn't in second statement, p is preceded by * symbol?
Q: How to print address of a variable
Q: format specifier?
Q: O/P of void fun(const int *p){ *p=0; } int main(){ const int i=10; fun(&i); return 0; }

Q: If i is a variable & points to i. Then choose o/p a) *p b) *&p c) &p d) *i e) *&i

Q: Pointer Arithmetic addition
Q: Simplified eg
Q: Compare a[3] and 3[a]
Q: TIP about address

Scanned with CamScanner

Page
Date

Q. Pointer authmetic subtraction
Q. eg
Q. Undefined behaviour - 1 defn
Q. eg
Q. " " - 2 defn
Q. eg
Q. Pointer Authmetic - Increment & decrement
Q. Post increment - eg
Q. Pre " - eg
Q. Pre & Post decrement eg
Q. How pointers compared
Q. Necessary condition to " pointers
Q. O/P dependson -
Q. O/P of

```
int main(){
int a[] = {1, 2, 3, 4, 5, 6};
int *p = &a[3];
int *q = &a[5];
printf("%d\n", p<=q);
printf("%d\n", p>=q);
q = &a[3];
printf("%d", p==q);
return 0;
}
```

Q. Sum of elements of array using pointer
Q. Using array name as pointer.
Q. *(a+i) meaning
Q. Rewrite sum of array code
Q. where to be careful

```
int main(){
int a[] = {11, 22, 33, 5, 24};
printf("%p", a+4);
```

Scanned with CamScanner

Q. Reverse array using pointers
Q. Passing array name as argument to
Q. Diff b/w row major & column major
Q. Search major (User)
Q. Print 2d array without pointers
Q. " " with "
Q. Consider char a[100][100]. Assume that
Draft 2022 memory is byte addressable &
that the array is stored starting
from the memory address 0, the
address of a[40][50] is
a) 4040 b) 4050 c) 5040 d) 5050

Q. Write general formula of row Major
Draft 2015 Q. What is o/p of following c code? Assume
that address of x is 2000 (in decimal)
& an integer requires 4 bytes of memory

```
#include <stdio.h>
int main(){
unsigned int x[4][3]={{1,2,3},{4,5,6},{7,8,9},{10,11,12}};

printf("%u, %u, %u", x+3, *(x+3), *(x+2)+3);
```

a) 2036, 2036, 2036 b) 2012, 4, 2004 c) 2038, 10, 10
d) 2012, 4, 6

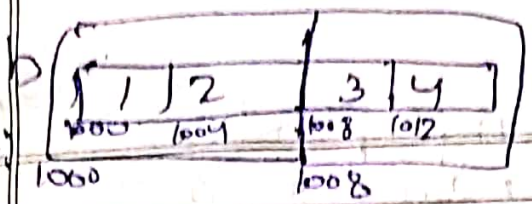
Q.
```
#include <stdio.h>
int main(){
int a[5]={1,2,3,4,5};
int *p=a;
printf("%d", *p);
return 0;
}
```
→ Mutake

Q. int (*p)[5]=&a ; Explain procedure steps

Scanned with CamScanner

Page
Date

Q.



a) *b
b) &*b
c) p
d) *p
e) **p

Q. Another legal def of 2D array

```
#include<stdio.h>
int main(){
int a[][3] = {1,2,3,4,5,6};
int (*ptr)[3] = a;
printf("%d %d", (*ptr)[1], (*ptr)[2]);
++ptr;
printf("%d %d", (*ptr)[1], (*ptr)[2]);
}
```

a) 2356
b) 2345
c) 4500
d) None

Q. (*ptr)[3] = a, *ptr[1] = , *ptr[2] =
*ptr =
*ptr[1] =

Q. O/P of

```
void f(int *p, int *q){
    p = q;
    *p = 2;
}
int c = 0, j = 1;
int main(){
    f(&i, &j);
    printf("%d %d\n", i, j);
}
```

a) 22
b) 21
c) 01
d) 02

Q. Value printed by

```
#include<stdio.h>
int f(int *a, int n){
    if(n<=0) return 0;
    else if(*a%2==0) return *a + f(a+1, n-1);
    else return (*a - f(a+1, n-1));
}
```

Scanned with CamScanner

Page 100
Date

```
int main()
{
  a[] = {2, 12, 7, 13, 4, 11, 6};
  printf("%d", f(a, 6));
  getchar();
  return 0;
}
```

a) 9 b) 5
c) 15 d) 9

Gate 2008

Q- int f(int x, int *py, int **ppz) {
int y, z;
**ppz += 1;
z = **ppz;
*py += 2;
y = *py;
x += 3;
return x+y+z; }
void main() {
int c, *b, **a;
c = 4; b = &c; a = &b;
printf("%d", f(c, b, a));
}

a) 18
b) 19
c) 21
d) 22

Gate 2008 - printab is called twice in main() then

void swap(int *x, int *y) {
static int *temp;
temp = x;
x = y;
y = temp; }
void printab() {
static int i, a = -3, b = -6;
i = 0;
while (i <= 4) {
if ((i++) % 2 == 1) continue;
a = a + i;
b = b + i;
} swap(&a, &b); printf("a = %d, b = %d\n", a, b); }

a) a=0, b=3
a=0, b=3
b) a=3, b=0
a=12, b=9
c) a=3, b=6
a=3, b=8
d) a=6, b=3
a=15, b=0

Scanned with CamScanner

Page
Date

Q contents of b at end

```
#include<stdio.h>
int main(){
int i,j;
char a[2][3]={{'a','b','c'},
              {'d','e','f'}};
char b[3][2],j;
char *p=*b;
for(i=0;i<2;i++){
  for(j=0;j<3;++j)
*(p+2*j+i)=a[i][j];
}}
```

a) a b / b) a d
c d / b e
e f / c f

c) a c / d) a e
e b / d c
d f / b f

Q. String literals / other name — def.

Eg

Placeholder

quotes type

What if we want to print in next line — \n

— 2

— 3

O/P of 1
2
3

Problem in 2

1. first argument in printf & scanf is always
2. how string literals are stored
3. visualize Earth
4. Total memory
5. ; must not be confused with ,
6. #5 (1): values of both
7. what we are actually passing to printf/scanf

Scanned with CamScanner

Both: printf & scanf expect — a
Passing "Earth" is equivalent to argument
Assigning string literal to a pointer
Subscript char *ptr = "Hello World";

"Hello"[0] = 'H';
"Hello"[1] =

Imp point
char *ptr = "Hello";
*ptr = 'M';

Other name of string literals
Type of memory allocated
char Pointer
String literal V/s character Constant
"H" & 'H'
printf("\n");
printf('\n');

Declaring a string variable.
eg.
Reserve character? Why
What if not done
Initializing syntax.
What it seems like but not?
2 ways of char → hello
We can/can't modify a string literal
We can/can't modify a char array
Short length initializer
char s[2] = "Hello";
Long length initializer
Why error/warning
Equal length initializer.
Allowed/Not

Page
Date

Q. Omitting the length/No length indicator
eg
Writing string using printf - eg
Format specifier in printf
a) Part of string from start
b) ,, ,, ,, any part

char *ptr = "Hello World!";
printf("%.5s", ptr);
printf("%5s", ptr);

Q. Write string using puts()
Where puts() declared
Q. Syntax & specialiaty of puts()
Q. O/P of char *s = "Hello";
puts(s);
puts(s);

Q. Reading string using scanf
Q. why not address of (a) used
char a[10];
printf("Enter the string: \n");
scanf("%s", a);
printf("%s", a); I/P: You are most welcome

Q. Why only You in O/P?
Q. Where gets → lib
Q. Gets function
char a[10];
printf("Enter the string \n");
gets(a);
printf("%s", a);
I/P: You are most welcome

Page 104
Date

Q1. Disadvantage of both gets() & scanf()

Q. What is buffer overflow & crash

Q. How to set limit in # of I/P character in scanf

Q. Syntax

Q. char a[10];
printf("Enter the string: \n");
scanf("%0.9s", a);
printf("%s", a);

I/P = you are most welcome

Q. Why gets() is unsafe

Q. What is advised.

Scanned with CamScanner